

Heimdall

Sees model retirements coming, proves the replacement on your real traffic, and cuts over before anything breaks.

Bifrost track Build a Feature the Open-Source Gateway Is Missing

Tavily track Build a Stale-Answer Detector for AI Applications

The first you hear of a retirement is a 404

The gateway knows

Which models you depend on, how much traffic each carries, and which applications send it.

The provider announces

A row in a documentation table, a changelog line, an email to whoever created the account.

Nothing connects them

Bifrost classifies a retired model as model_gone only after requests have started failing.

Heimdall answers, in order: what do we depend on, what is being retired and when, is the replacement safe for our traffic, and how do we switch without an outage.

Four steps, each on a Bifrost feature

1

Inventory

Which models carry traffic, and for which virtual keys.

Gateway log aggregates

2

Radar

Retirement notices from the provider's own documentation.

Tavily, through a Bifrost relay

3

Rehearsal

Real logged requests replayed against the replacement.

Logs, virtual key, budget

4

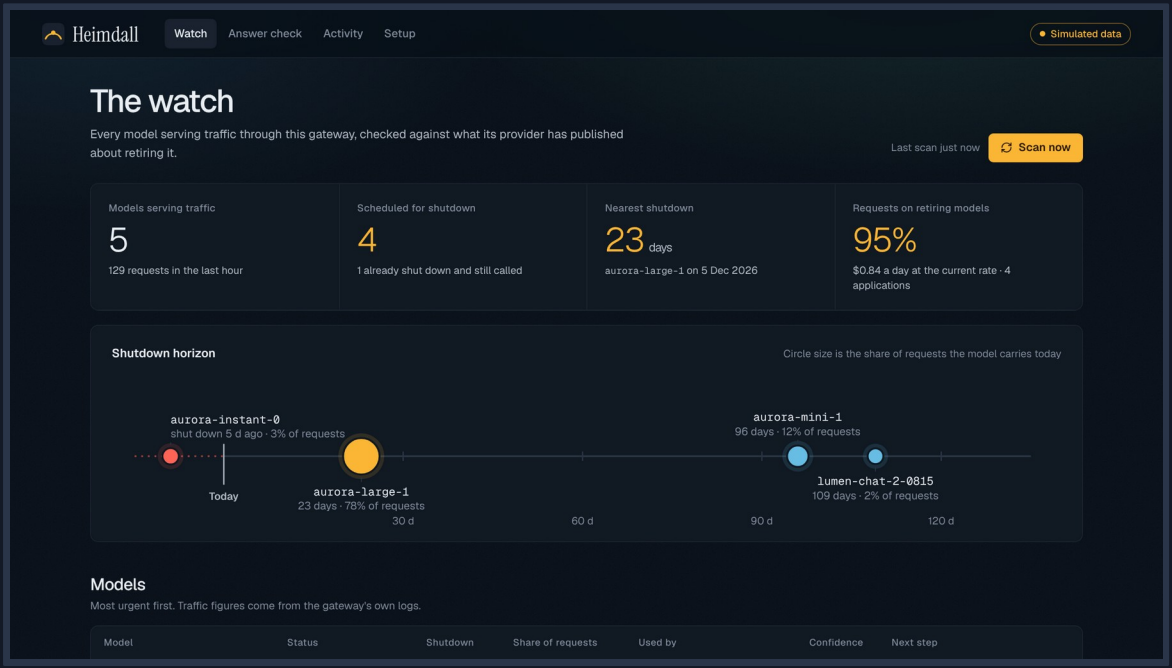
Cutover

Safety net, canary, cut over. One reversible rule.

Routing rules

No fork, no patched binary, no plugin to compile. It runs beside the official maximhq/bifrost image and uses only public endpoints.

The watch: time left against traffic at stake



5 models in traffic

Read from the gateway's own logs.

4 scheduled for shutdown

One carries most of the requests and has 23 days left.

1 already shut down

A forgotten batch job is still calling it.

Simulated scenario: invented providers and pages behind a real Bifrost gateway.

The model points at the text. Heimdall decides

1. **A quote must be in the source**
   A paraphrase is discarded, and the discard is shown.
2. **A date must be in the quote**
   A date the model remembers is dropped.
3. **Confidence comes from the source**
   Never from how sure the model sounded.
4. **The provider outranks everyone**
   Disagreement is shown, not hidden.



Each rule has a test with a model that misquotes, invents a date, or reads a neighbouring model's row.

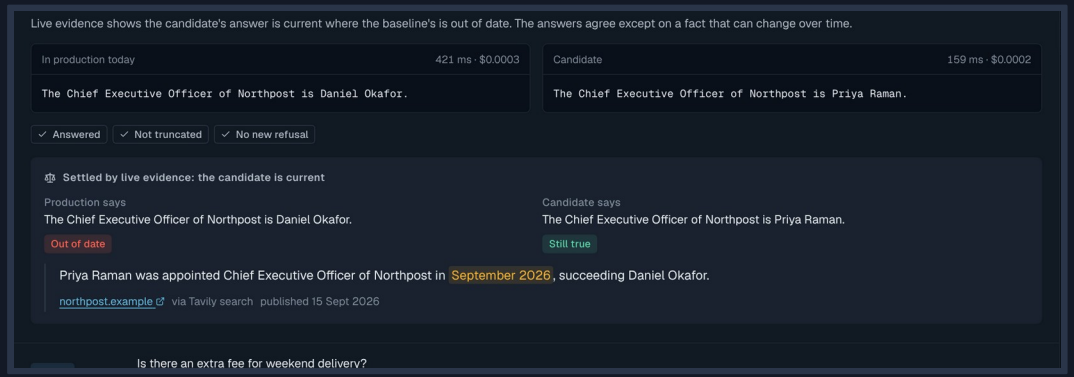
Rehearsal: your traffic, not a benchmark

40
real requests replayed

3
regressions, found by a parser

−35%
cost on this traffic

6
answers more current



The stale-answer detector, at work

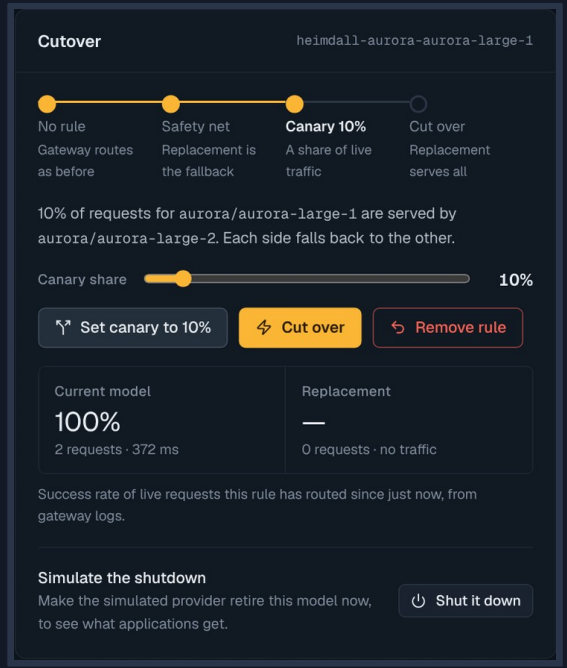
When the two models disagree on a fact that changes over time, neither is assumed right. Both claims are checked against the web today.

Figures from one rehearsal in the simulated scenario. Production is untouched; replays run under a budgeted virtual key.

Cutover is one routing rule

Safety net	Old model 100%, replacement as fallback. A provider shutdown is served, not failed.
Canary	A weighted split on live traffic. Health is measured from gateway logs.
Cut over	Replacement 100%. Applications keep the old model name.

Removing the rule restores the gateway exactly.



Tested on a real gateway: with the model shut down upstream, the same request returns 404 without the rule and an answer with it.

One boundary: everything goes through Bifrost

Bifrost

Logs and aggregates inventory, sampling, canary health

Routing rules safety net, canary, cut over, rollback

Virtual keys and budgets Heimdall's own spend is capped

Relays Tavily sits behind the gateway

Logging headers its own traffic never counts as demand

Model API shutdown_date as first-party evidence

MCP gateway three tools for agents

Tavily

Extract

Provider lifecycle pages, in full.

Search, official domains

Per model, restricted to the provider's documentation.

Search, open web

Corroboration and early warning.

Search, per claim

The stale-answer detector.

Heimdall holds one virtual key and the gateway's admin login. No provider keys, no Tavily key.

What is verified, and what is not yet

Verified

- End-to-end suite against a real Bifrost binary: inventory, radar, rehearsal, all cutover stages, answer check, MCP registration.
- The container on the official maximhq/bifrost image.
- Grounding rules, against a model scripted to misquote and invent dates.
- Live-mode configuration, up to the provider boundary.

Not yet

- Real model providers and the real Tavily API. The build environment had no route to them.
- The Tavily client is tested against the request and response shapes of Tavily's SDK.
- In the simulator, the analyst is a rule-based stand-in, not a language model.
- The first run with real keys is the first test of those two boundaries.

Try it in one command

```
docker build -t heimdall . && docker run --rm -p 7860:7860 heimdall
```

No keys: simulator mode

A real Bifrost gateway with simulated providers, a simulated web and four simulated applications. Shut a model down and watch the safety net hold.

Add keys: live

A provider key, TAVILY_API_KEY and HEIMDALL_ADMIN_TOKEN. Restart, and the same container runs against real providers and the real web.

github.com/angarkartanmay-ops/GlobalAiHack • **Live demo: heimdall-sim.vercel.app**